

Before starting with Coding

Electronics Project 2: Radar

Abstract

Ever eager to see a first result on the screen, the temptation is big to just start up the development environment and begin with coding. However, taking some time to sketch the user interface, the data model, the algorithms, and their interdependencies on paper speeds up coding and helps getting it right from scratch.

(c) Hanspeter Hochreutener, hhrt@zhaw.ch, 19.03.2021

Table of Content

Abstract	1
1. Model - View - Controller	2
1.1. Design procedure	2
2. Model	2
2.1. Files (modules)	3
2.2. .h files and namespaces (prefixes)	3
2.3. Defines (constants)	3
2.4. Variables	4
2.5. Functions	4
3. View	4
4. Controller	4
5. Ready?	4

1. Model - View - Controller

<https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller> gives the following definition:

Model

The central component of the pattern. It is the application's dynamic data structure, independent of the user interface.[5] It directly manages the data, logic and rules of the application.

View

Any representation of information such as a chart, diagram or table. Multiple views of the same information are possible, such as a bar chart for management and a tabular view for accountants.

Controller

Accepts input and converts it to commands for the model or view.

The next sections show how this programming paradigm may be applied to the radar project.

1.1. Design procedure

It is well known that it is easier to make a creative draft with pencils on paper.

- Take three sheets of paper, write “Model”, “View” and “Controller” each on one sheet.
- Draw sketches, diagrams, flow charts, name activities and data entities, define interdependencies, jot down whatever seems related.
- Define the most important variables.
- Define the most important functions.
- Group functions and variables in modules with well-defined scopes.
- Take new sheets of paper and try different groupings. Discard the ones with more dependencies and keep the more elegant.

2. Model

The model includes measuring the signals, pre-processing the samples, calculating the FFT, determine the speed of the object. Calculation of spectrograms or speed patterns would also be handled in the model. The model is also responsible for the different modes of operation like single or continuous measurement, control the time sequences for Doppler or FMCW modes, etc.

Remember: Displaying the results is a task of the view not the model, and the controller is in charge for handling user inputs.

2.1. Files (modules)

The model should be further split into different modules with an .h and a .c file for each. For the radar project this could be done like this:

- **measuring.h and .c**
Includes everything that is needed for sampling the signals till storing the values in variables (arrays).
Functions for configuring ADC, timer and DMA, signal sampling, conversion to data types suitable for processing. For FMCW control of the DAC, switching of sample frequency.
Variables for sampling and converted data.
- **fft.h and .c**
The fft is used in a similar way for the doppler and the FMCW radar. In order to avoid code redundancy, it is sensible to put those common functions and variables in a separate module.
- **doppler.h and .c**
Functions and variables for the calculation of the speed(s). A spectrogram $\text{speed} = f(\text{time})$ would also be computed here.
- **fmcw.h and .c**
Functions and variables for the calculation of range(s). A spectrogram $\text{range} = f(\text{time})$ would also be computed here.
- **range_doppler.h and .c**
Range-doppler-radar is an extension of fmcw in such that the speed can be calculated from a series of fmcw range measurements. The extra data and algorithm may be put in this separate module to keep the fmcw module short and concise.

2.2. .h files and namespaces (prefixes)

As the members declared in a .h file can be included in other modules, a prefix is helpful to avoid name conflicts. As an example the names of the defines, variables and functions in **measuring.h** could start with the prefix **MEAS_**

Of course, all this applies also to the modules of the view and the controller.

2.3. Defines (constants)

In measuring.h (as an example) the following constant might be defined:

```
#define MEAS_ADC_DAC_RES      12    ///< Resolution of ADC and DAC
```

MEAS_ADC_DAC_RES is probably used in other modules and must therefore be defined in the .h file, whereas the following **ADC_CLOCK** is strictly used privately and must therefore be put in the file measuring.c .

```
#define ADC_CLOCK             84000000    ///< APB2 peripheral clock frequency
```

2.4. Variables

In measuring.h (as an example) the following variable might be declared:

```
extern float32_t MEAS_ADC_SAMPLES[2*MEAS_N];    // Converted I and Q values
/** The order of the samples is [I0, Q0, I1, Q1, I2, ...]
 * which represents MEAS_N complex samples  $In+jQn$  */
```

However, the following variable with the raw ADC values is used only privately and must therefore be put in the file measuring.c .

```
static uint32_t ADC_samples[MEAS_N];    ///< Buffer with ADC values
/** Converted data from ADC1 and ADC2 are packed into a 32-bit register
 * in this way: <b> ADC_CDR[31:0] = ADC2_DR[15:0] | ADC1_DR[15:0] */
```

2.5. Functions

In measuring.h (as an example) the following function might be declared:

```
void MEAS_ADC_DAC_init(void);            ///< initialise ADC and DAC
```

But not:

```
void DMA2_Stream4_IRQHandler(void);      ///< IRQ handler for DMA2 stream4
```

3. View

This is everything that uses the display: Draw curves, print numbers, show the menu.

The view might be split in several modules as explained above for the model.

4. Controller

The controller contains everything that has to do with user input. For the radar prototype this is basically the touchscreen with the menu items and the push button.

The controller might be split in several modules as explained above for the model.

5. Ready?

With the well-defined structure for the algorithms, the variables and the user interface it is time to start up the development environment. Isn't it?

You might find it useful to take a look at the coding guide lines summarized in these slides <https://ennis.zhaw.ch/files/CodingGuidelines.pdf>